

BANCO DE DADOS

com Microsoft SQL Server

AULA 05

DML Parte II — SELECT e WHERE

Consultando dados do SISTEMA_VENDAS

SELECT

WHERE

Op. Relacionais

AND / OR / NOT

DISTINCT

Alias

TOP / OFFSET

ORDER BY



Objetivos da Aula 05



Dominar a estrutura completa do comando SELECT



Renomear colunas e tabelas com Alias (AS)



Filtrar registros com WHERE e operadores relacionais



Criar colunas calculadas diretamente no SELECT



Combinar condições com AND, OR e NOT (e precedência!)



Limitar resultados com TOP e paginar com OFFSET/FETCH



Eliminar duplicatas com DISTINCT



Ordenar resultados com ORDER BY ASC e DESC



Revisão — Aula 04: INSERT

? Por que sempre listar colunas no INSERT?

✓ Se a tabela mudar, o INSERT sem colunas pode inserir dados errados silenciosamente

? Qual a diferença entre INSERT múltiplo e INSERTs separados?

✓ INSERT múltiplo VALUES(v1),(v2) é muito mais eficiente — menos round-trips ao banco

? O que faz SCOPE_IDENTITY()?

✓ Retorna o último IDENTITY gerado na sessão atual — seguro mesmo com triggers

? O que é SELECT INTO?

✓ Cria uma nova tabela e insere dados em um único comando — ótimo para backups

? Qual a ordem correta para inserir dados?

✓ Primeiro tabelas sem FK (CATEGORIAS, CLIENTES), depois as com FK (PRODUTOS, PEDIDOS, ITENS)

Anatomia do SELECT — Estrutura Completa

Estrutura completa do SELECT

```
SELECT    [DISTINCT] col1, col2, ...
```

```
FROM      tabela [AS alias]
```

```
WHERE     condição
```

```
GROUP BY  colunas
```

```
HAVING    condição_grupo
```

```
ORDER BY  coluna [ASC|DESC]
```

```
OFFSET n ROWS
```

```
FETCH NEXT m ROWS ONLY;
```

O QUÊ buscar

colunas, expressões,
*

DE onde

tabela ou view

FILTRO de linhas

condição booleana

AGRUPAR por

Aula 09

FILTRO de grupos

Aula 09

ORDENAR

resultado final

PULAR N linhas

paginação

PEGAR M linhas

paginação



Ordem de EXECUÇÃO (≠ escrita): FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → OFFSET/FETCH

SELECT — Primeiras Consultas no SISTEMA_VENDAS

```
USE SISTEMA_VENDAS;

-- 1. Todos os registros (evite SELECT * em produção!)
SELECT * FROM CLIENTES;

-- 2. Colunas específicas (boa prática)
SELECT nome, email, cidade, estado
FROM   CLIENTES;

-- 3. Colunas com ALIAS (renomear para exibição)
SELECT nome           AS [Nome do Cliente],
       cidade         AS Cidade,
       estado         AS UF,
       data_cadastro  AS [Membro desde]
FROM   CLIENTES;

-- 4. Coluna calculada
SELECT nome_produto,
       preco,
       estoque,
       preco * estoque AS valor_em_estoque,
       preco * 0.90    AS preco_com_desconto_10pct
FROM   PRODUTOS;

-- 5. TOP N: primeiros registros
SELECT TOP 3 nome_produto, preco FROM PRODUTOS ORDER BY preco DESC;
```

WHERE — Operadores Relacionais

Operadores disponíveis:

=

Igual a

```
estado = 'SP'
```

!=

Diferente (alt.)

```
status != 'PENDENTE'
```

>=

Maior ou igual

```
estoque >= 10
```

<=

Menor ou igual

```
total <= 500
```

<>

Diferente de

```
status <> 'CANCELADO'
```

>

Maior que

```
preco > 100
```

<

Menor que

```
salario < 3000
```

BETWEEN

Entre (inclusivo)

```
preco BETWEEN 50 AND 200
```



BETWEEN é inclusivo nos dois extremos: BETWEEN 50 AND 200 equivale a >= 50 AND <= 200. Funciona com números, datas e texto.



WHERE — Consultas Reais no SISTEMA_VENDAS

```
-- Clientes de São Paulo
SELECT nome, email, cidade FROM CLIENTES
WHERE estado = 'SP';

-- Produtos com preço acima de R$ 500
SELECT nome_produto, preco, estoque
FROM PRODUTOS WHERE preco > 500.00;

-- Pedidos não cancelados
SELECT id_pedido, id_cliente, status, total
FROM PEDIDOS WHERE status <> 'CANCELADO';

-- Produtos em faixa de preço (BETWEEN)
SELECT nome_produto, preco FROM PRODUTOS
WHERE preco BETWEEN 50.00 AND 300.00
ORDER BY preco;

-- Pedidos com total maior que a média (subquery no WHERE)
SELECT id_pedido, status, total FROM PEDIDOS
WHERE total > (SELECT AVG(total) FROM PEDIDOS);

-- Clientes com ID entre 3 e 6
SELECT id_cliente, nome FROM CLIENTES
WHERE id_cliente BETWEEN 3 AND 6;

-- BETWEEN com datas
SELECT id_pedido, data_pedido, total FROM PEDIDOS
WHERE data_pedido BETWEEN '2025-01-01' AND '2025-12-31';
```

Operadores Lógicos — AND, OR e NOT

Como cada operador funciona:

AND

AMBAS as condições devem ser TRUE

A	B	A AND B
TRUE	TRUE	✓ TRUE
TRUE	FALSE	✗ FALSE
FALSE	TRUE	✗ FALSE
FALSE	FALSE	✗ FALSE

OR

PELO MENOS UMA condição deve ser TRUE

A	B	A OR B
TRUE	TRUE	✓ TRUE
TRUE	FALSE	✓ TRUE
FALSE	TRUE	✓ TRUE
FALSE	FALSE	✗ FALSE

NOT

INVERTE o resultado da condição

A	NOT A
TRUE	✗ FALSE
FALSE	✓ TRUE

⚠ Precedência: NOT > AND > OR | Sempre use parênteses para deixar explícito e evitar bugs!



AND, OR e NOT — Exemplos Práticos

```
-- AND: TODAS as condições devem ser verdadeiras
SELECT nome, cidade, estado FROM CLIENTES
WHERE estado = 'SP' AND cidade = 'São Paulo';
-- OR: PELO MENOS uma condição verdadeira
SELECT nome, estado FROM CLIENTES
WHERE estado = 'SP' OR estado = 'RJ';
-- Mesmo resultado com IN (mais legível):
WHERE estado IN ('SP','RJ');
-- NOT: inverte a condição
SELECT nome_produto, preco FROM PRODUTOS
WHERE NOT id_categoria = 1; -- Tudo exceto Eletrônicos
-- Combinando AND e OR — USE PARÊNTESES!
-- Produtos de Roupas ou Esportes com preço < R$ 200
SELECT nome_produto, preco, id_categoria FROM PRODUTOS
WHERE (id_categoria = 2 OR id_categoria = 5) -- parênteses!
AND preco < 200.00;
-- ⚠ SEM parênteses: resultado ERRADO (bug silencioso!)
-- WHERE id_categoria = 2 OR id_categoria = 5 AND preco < 200
-- SQL lê: id_cat=2 OR (id_cat=5 AND preco<200) -- diferente!
-- AND + OR + NOT combinados
SELECT nome, estado FROM CLIENTES
WHERE (estado = 'SP' OR estado = 'RJ')
AND NOT email IS NULL; -- ou: AND email IS NOT NULL
```



DISTINCT e Alias — Refinando o Resultado

```
-- DISTINCT: elimina linhas duplicadas no resultado

-- Quais estados temos clientes? (sem DISTINCT = duplicatas)
SELECT DISTINCT estado FROM CLIENTES ORDER BY estado;

-- Quais categorias têm produtos cadastrados?
SELECT DISTINCT id_categoria FROM PRODUTOS;

-- DISTINCT em múltiplas colunas (combinação única)
SELECT DISTINCT estado, cidade FROM CLIENTES ORDER BY estado, cidade;

-- COUNT com DISTINCT: quantos estados distintos?
SELECT COUNT(DISTINCT estado) AS total_estados FROM CLIENTES;

-- ALIAS: renomear colunas e tabelas
SELECT nome          AS 'Nome Completo',
       email         AS 'E-mail',
       cidade        AS Cidade,
       estado        AS UF
FROM   CLIENTES AS c  -- alias de tabela (útil em JOINS!)
WHERE  c.estado <> 'DF';

-- Alias de tabela (forma curta, sem AS)
SELECT p.nome_produto, p.preco
FROM   PRODUTOS p     -- alias sem AS
WHERE  p.estoque > 0;
```

Colunas Calculadas — SELECT como Calculadora

O SELECT pode calcular expressões diretamente, criando colunas virtuais no resultado. Essas colunas não existem na tabela — são calculadas em tempo de execução.

Aritmética

```
SELECT nome_produto, preco,  
       preco * 1.12 AS [Com 12% imposto],  
       preco * 0.90 AS [Com 10% desconto],  
       preco * estoque AS valor_em_estoque  
FROM   PRODUTOS;
```

String (texto)

```
SELECT CONCAT(nome, ' - ', cidade, '/', estado)  
       AS identificacao,  
       LEN(nome) AS tamanho_nome  
FROM   CLIENTES;
```

Data e hora

```
SELECT nome,  
       DATEDIFF(DAY, data_cadastro,  
               GETDATE()) AS dias_cadastrado,  
       FORMAT(data_cadastro, 'dd/MM/yyyy')  
       AS cadastro_br  
FROM   CLIENTES;
```

Condicional (CASE)

```
SELECT nome_produto, estoque,  
       CASE  
         WHEN estoque = 0 THEN 'Sem estoque'  
         WHEN estoque < 10 THEN 'Estoque baixo'  
         ELSE 'Disponível'  
       END AS situacao  
FROM   PRODUTOS;
```



ORDER BY — Ordenando Resultados

```
-- ORDER BY: ordena o resultado (sempre o último passo!)
-- ASC = crescente (padrão) | DESC = decrescente

-- Produto mais caro primeiro
SELECT nome_produto, preco FROM PRODUTOS
ORDER BY preco DESC;

-- Ordenação por múltiplas colunas
SELECT nome, estado, cidade FROM CLIENTES
ORDER BY estado ASC, cidade ASC, nome ASC;

-- Ordenar por alias (definido no SELECT)
SELECT nome_produto, preco * estoque AS valor_total
FROM PRODUTOS
ORDER BY valor_total DESC; -- alias funciona no ORDER BY!

-- Ordenar por posição da coluna (evite - pouco legível)
SELECT nome_produto, preco FROM PRODUTOS
ORDER BY 2 DESC; -- 2 = segunda coluna = preco

-- NULL: vão por último no ASC, primeiro no DESC (padrão SQL Server)
SELECT nome, email FROM CLIENTES ORDER BY email ASC;

-- Clientes sem email aparecem no final

-- TOP com ORDER BY: os 3 pedidos de maior valor
SELECT TOP 3 id_pedido, total, status FROM PEDIDOS
ORDER BY total DESC;
```

TOP e OFFSET/FETCH — Limitando e Paginando Resultados

Comparação das abordagens:

TOP — Simples e direto

```
-- Primeiros 5 produtos mais caros
SELECT TOP 5 nome_produto, preco
FROM   PRODUTOS ORDER BY preco DESC;

-- TOP com PERCENT
SELECT TOP 30 PERCENT nome_produto
FROM   PRODUTOS ORDER BY preco DESC;

-- WITH TIES: inclui empates
SELECT TOP 3 WITH TIES
       nome_produto, preco
FROM   PRODUTOS ORDER BY preco DESC;
```

OFFSET/FETCH — Padrão ANSI

```
-- Página 1: primeiros 3 (OFFSET 0)
SELECT nome_produto, preco
FROM   PRODUTOS ORDER BY preco DESC
OFFSET 0 ROWS
FETCH NEXT 3 ROWS ONLY;

-- Página 2: pular 3, pegar próximos 3
SELECT nome_produto, preco
FROM   PRODUTOS ORDER BY preco DESC
OFFSET 3 ROWS
FETCH NEXT 3 ROWS ONLY;

-- Fórmula: OFFSET (pagina-1)*tamanho ROWS
```

✅ OFFSET/FETCH requer ORDER BY obrigatório. É o padrão ANSI SQL e funciona em SQL Server 2012+. Para APIs e apps web com listas paginadas, é a abordagem recomendada.



Consultas Combinadas — Casos Reais de Negócio

```
-- CASO 1: Relatório de produtos para campanha
-- (Roupas e Esportes com estoque alto e preço acessível)

SELECT nome_produto AS produto,
       preco        AS 'Preço (R$)',
       estoque      AS 'Em estoque',
       preco * 0.85 AS 'Preço promo (-15%)'

FROM   PRODUTOS
WHERE  id_categoria IN (2, 5)
AND    estoque > 20
AND    preco <= 300.00
ORDER  BY preco ASC;

-- CASO 2: Clientes para campanha de e-mail (SP e RJ com e-mail)

SELECT nome          AS [Nome do Cliente],
       email         AS [E-mail],
       cidade        AS Cidade,
       DATEDIFF(DAY, data_cadastro, GETDATE()) AS [Dias cadastrado]

FROM   CLIENTES
WHERE  estado IN ('SP','RJ')
AND    email IS NOT NULL
ORDER  BY dias_cadastrado DESC;

-- ⚠ erro! alias não pode ser usado no ORDER BY neste caso

-- Correto:

ORDER  BY DATEDIFF(DAY, data_cadastro, GETDATE()) DESC;

-- CASO 3: Top 3 pedidos de maior valor (excluindo cancelados)

SELECT TOP 3 id_pedido, id_cliente, status,
       FORMAT(total,'C','pt-BR') AS total_formatado

FROM   PEDIDOS WHERE status <> 'CANCELADO'
ORDER  BY total DESC;
```



Exercícios Propostos — Aula 05

01

Básico

Liste todos os produtos com preço entre R\$50 e R\$500 usando BETWEEN, ordenados do mais barato ao mais caro. Mostre nome, preço e estoque.

02

Básico

Liste os 3 pedidos de maior valor (excluindo CANCELADOS). Mostre id_pedido, id_cliente, status e total formatado. Use TOP e ORDER BY.

03

Médio

Mostre clientes de SP ou RJ que têm email cadastrado, ordenados por nome. Use aliases nas colunas: 'Nome do Cliente', 'E-mail', 'Estado'. Demonstre com e sem parênteses para mostrar a diferença.

04

Médio

Crie uma consulta com colunas calculadas para PRODUTOS: nome, preço original, preço com 8% de desconto, preço com 15% de aumento, e uma coluna CASE classificando como 'Barato' (<R\$100), 'Médio' (R\$100-R\$500) ou 'Caro' (>R\$500).

05

Avançado

Implemente paginação de produtos ordenados por preço DESC: mostre a Página 1 (3 produtos) e a Página 2 (próximos 3) usando OFFSET/FETCH. Depois, mostre o mesmo resultado usando TOP com subquery.



Respostas Comentadas — Exercícios 03 e 04

```
-- EX 03: Clientes SP ou RJ com email — COM parênteses (correto)
SELECT nome          AS [Nome do Cliente],
       email         AS [E-mail],
       estado        AS [Estado]
FROM   CLIENTES
WHERE  (estado = 'SP' OR estado = 'RJ')  -- parênteses essenciais!
AND    email IS NOT NULL
ORDER  BY nome;
```

```
-- EX 04: Colunas calculadas com CASE
SELECT nome_produto      AS Produto,
       preco             AS [Preço Original],
       ROUND(preco * 0.92, 2) AS [Com 8% desconto],
       ROUND(preco * 1.15, 2) AS [Com 15% aumento],
       CASE
           WHEN preco < 100   THEN 'Barato'
           WHEN preco <= 500  THEN 'Médio'
           ELSE                'Caro'
       END               AS Classificacao
FROM   PRODUTOS
ORDER  BY preco;

-- ROUND(valor, 2): arredonda para 2 casas decimais
-- CASE WHEN ... THEN ... ELSE ... END: estrutura condicional
```




Desafio da Aula 05

Contexto: O gerente solicitou uma série de relatórios do SISTEMA_VENDAS para reunião de resultados. Você deve criar todas as consultas usando apenas SELECT, WHERE, ORDER BY e os recursos desta aula.

Relatórios solicitados:

R1

Catálogo completo de produtos com: nome, categoria (id), preço, estoque e valor total em estoque (preço × estoque). Ordenado por valor total DESC.

R2

Top 5 clientes mais antigos (pela data de cadastro). Mostre nome, estado, cidade e há quantos dias são clientes.

R3

Produtos com estoque crítico: estoque < 15 unidades, ordenados pelo estoque ASC. Inclua coluna CASE com 'Urgente' (≤5) ou 'Atenção' (6-14).

R4

Pedidos aprovados ou entregues com valor acima de R\$ 200. Mostre id, status, total e uma coluna calculada 'comissao_5pct' = total × 0.05.

R5

BÔNUS: Implemente o catálogo de produtos paginado (5 por página): gere as páginas 1, 2 e 3 com OFFSET/FETCH, ordenado por nome_produto.



Resumo — Aula 05: SELECT e WHERE



SELECT

Busca dados. Sempre liste as colunas — evite `SELECT *` em produção



WHERE

Filtra linhas. Sem `WHERE` = todos os registros. Sempre valide!



Op. Relacionais

`= <> != > >= < <= BETWEEN` — comparam valores. `BETWEEN` é inclusivo



AND / OR / NOT

`AND > OR` em precedência. Use parênteses para evitar bugs silenciosos!



DISTINCT

Elimina duplicatas no resultado. Pode ser lento em tabelas grandes



Alias (AS)

Renomeia colunas e tabelas. Use `[]` ou `"` quando tiver espaços



TOP / OFFSET

`TOP N`: primeiros N registros. `OFFSET/FETCH`: paginação padrão ANSI



ORDER BY

Ordena resultado. `ASC`(padrão)/`DESC`. Alias funciona, mas expressões repetir



Próxima Aula 06: Consultas Avançadas

Aula 06 — O que vem por aí:



LIKE e wildcards

% e _ para buscas por padrão de texto



BETWEEN

Intervalos com datas, textos e números



IN e NOT IN

Pertinência a listas — cuidado com NULL!



IS NULL / IS NOT NULL

Trabalhar corretamente com valores nulos



DISTINCT avançado

Combinações únicas e COUNT DISTINCT



Alias avançado

Renomear com expressões e subqueries



Para a próxima aula: conclua o desafio de relatórios e experimente criar suas próprias consultas no SISTEMA_VENDAS!